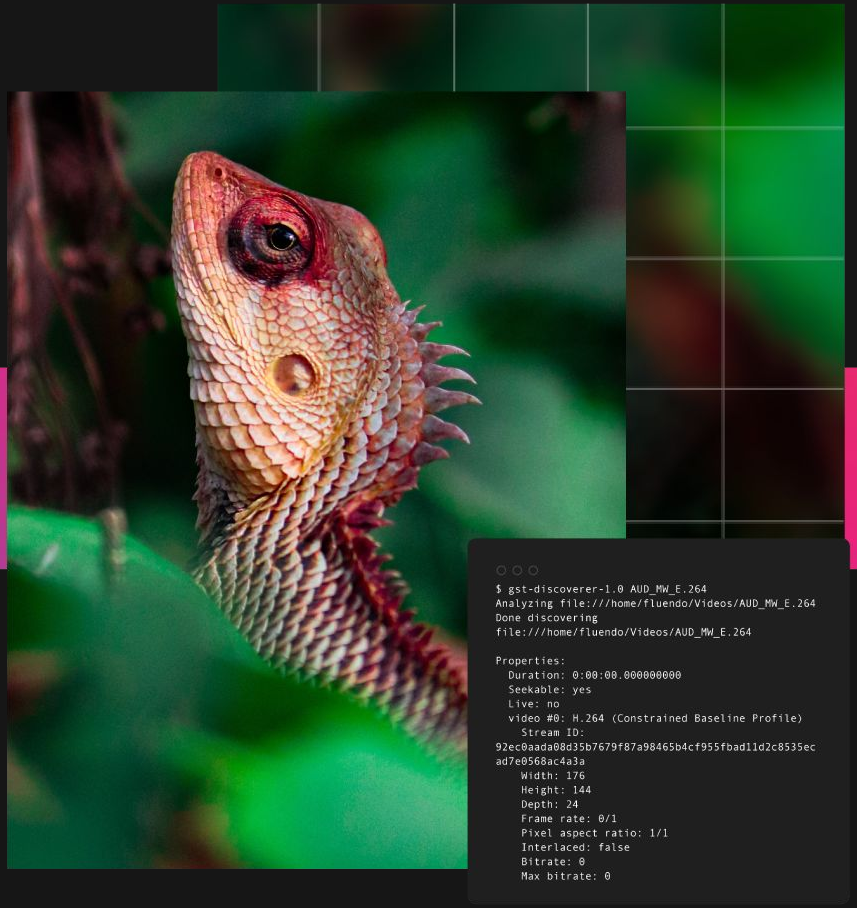
fluendo

GstWASM

GStreamer for the web



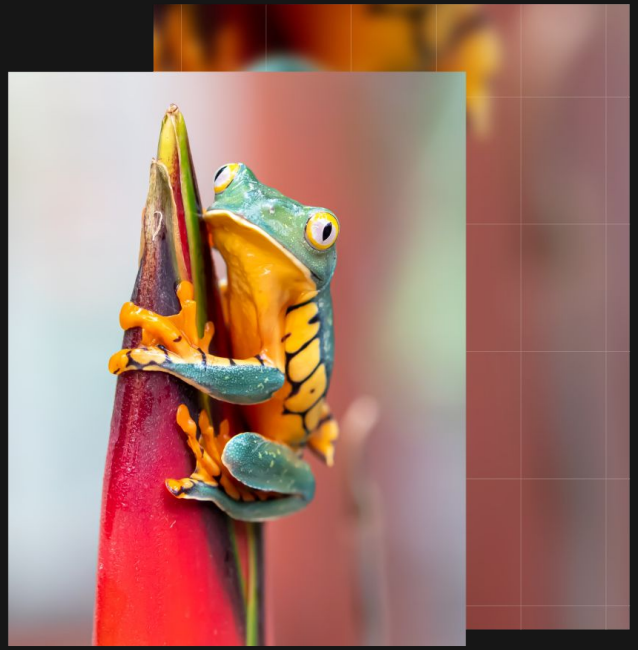
```
$ gst-discoverer-1.0 AUD_MW_E.264
Analyzing file:///home/fluendo/Videos/AUD_MW_E.264
Done discovering
file:///home/fluendo/Videos/AUD_MW_E.264

Properties:
  Duration: 0:00:00.000000000
  Seekable: yes
  Live: no
  video #0: H.264 (Constrained Baseline Profile)
    Stream ID:
92ec0aada08d35b7679f87a98465b4cf955fbad11d2c8535ec
ad7e0568ac4a3a
    Width: 176
    Height: 144
    Depth: 24
    Frame rate: 0/1
    Pixel aspect ratio: 1/1
    Interlaced: false
    Bitrate: 0
    Max bitrate: 0
```

Index

- Overview and motivation
- Migration
- Testing
- Contribution and future

Overview and motivation



Video codecs in the browser

- ActiveX
- Browser plugins (Totem plugin)
- Flash

Codecs external to the browser

- HTML5 Video
- MSE (Media Source Extension): DASH / HLS
- WebCodecs

Codecs internal to the browser (**missing extensibility**)

The problem

- New codecs “can not” be added to a browser under a seamless API
  - VVC, LC-EVC, Custom AV1, Custom H.265, etc
- How to handle patented / proprietary codecs in a monolithic architecture?
- The available codecs are backed up by the specification itself
  - Narrowed list of MIME types
- Modifying a browser to support new codecs is an overwhelming work

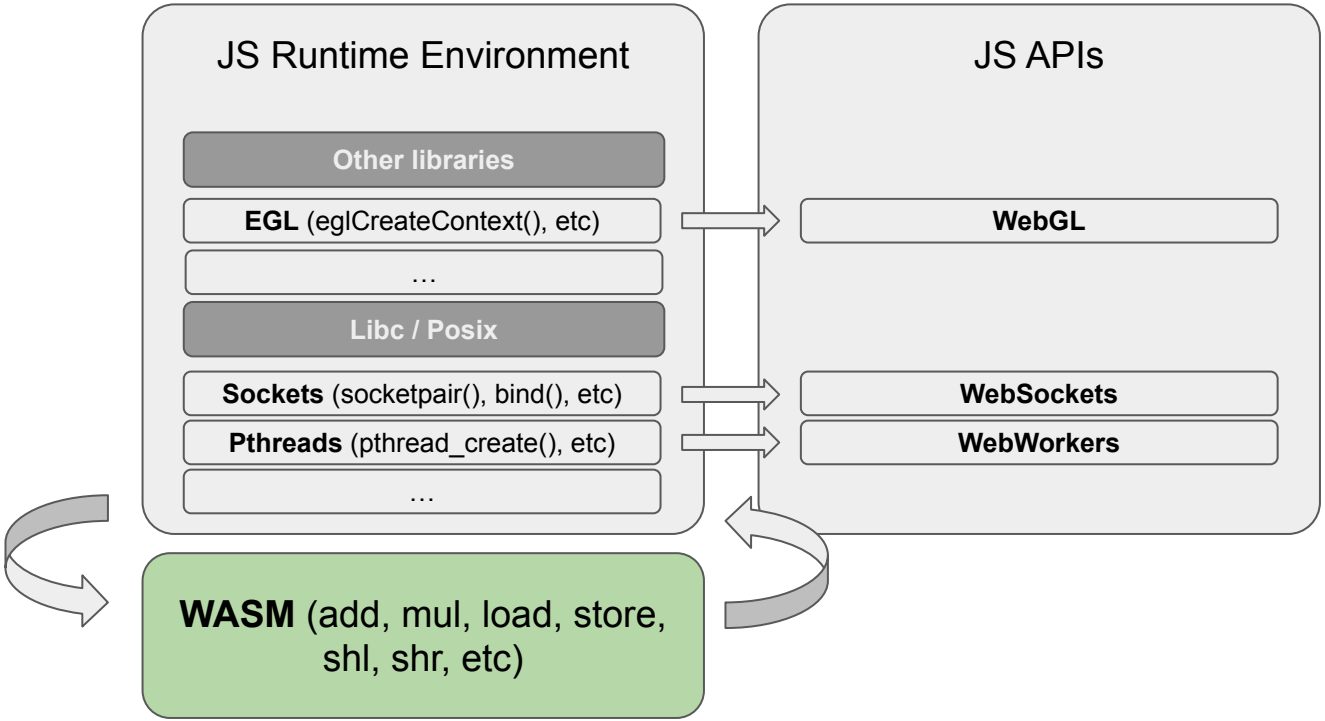
The goal

- Allow third party codecs through a seamless API: WebCodecs
- Bring legacy codecs into the web (MPEG-2, MPEG4 Part 2, WMV, VC1, etc)
- Bring cutting edge codecs into the web (VVC, LC-EVC, etc)
- Allow third party companies, like Fluendo, to bring their products into the web in a simpler way

WASM

- Stands for Webassembly
- Portable binary code
- Is standardized (W3C)
- LLVM has support for it as a target backend
- Does not directly access the standard Web interfaces (DOM, Canvas, Web Storage, WebRTC, etc.)

What WASM is not



JS Runtime Environment: Emscripten

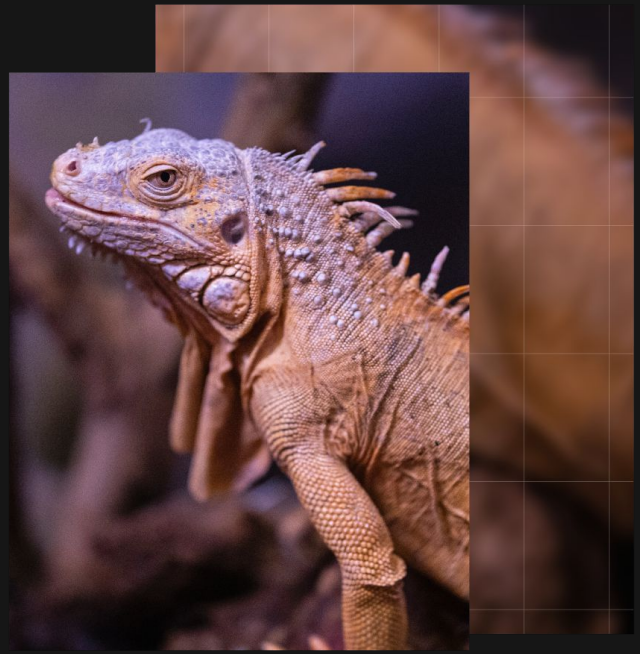
- Need Libc / POSIX compatibility
- Multiple implementations: WASIX, wajic, Emscripten, Cheerp, etc
- Not 100% compatible
- Chose Emscripten due the supported features to match native libraries into their web counterpart (i.e OpenGL -> WebGL)

The solution

What if we port GStreamer to the browser (WASM+Emscripten) and replace WebCodecs native JS interface with our own GStreamer based implementation to keep the compatibility, and at the same time we allow other codecs to be available in the browser as GStreamer plugins being browser agnostic?

Long story short: It’s doable and it works

Migration



Dependencies

- GLib and libffi ported to the target platform
- libvips project which depends on GLib was ported to WASM (wasm-vips)

Build system, dynamic linking, and dlopen()

- Emscripten features
  - Support of dynamic linking
  - Support of dynamic modules by setting up a file storage mechanism in a server
- For MVP, just build everything static
- **Misbehaviour of static building in GStreamer (June)**
  - Gst-full: dynamic library, everything “static”
  - Everything to be a .a file

Function pointer issues

- Undefined behavior if calling a C function pointer with different type (cast)
  - Common compilers targets are transparent to this
  - WASM aborts
- Compile with -Wbad-function-cast -Wcast-function-type
  - Huge amount of warnings -> aborts
- We modified the minimum code to have GStreamer initialized
- **What should be the next steps on this?**

GstBus, GstPollFD

- GstBus requires GstPollFD
- GstPollFd requires pipe(): pipe as an **inter-thread-communication?**
- No pipe() or unix sockets support on Emscripten

Threading

- Emscripten does support pthreads
- Emscripten needs to know beforehand how big the pool of “web workers” would be needed (linker flag -sPTHREAD_POOL_SIZE=8)
- **The number of threads used in a pipeline is application-dependent**

First results: in a browser

```
▶0:00:05.492255000    42   0x1a67d0 DEBUG          basetransform gstbasetransform.c:2174:default_generate_output:          test01.js:162
<glcolorbalance0> calling prepare buffer
▶0:00:05.492679000    42   0x1a67d0 DEBUG          basetransform gstbasetransform.c:1655:default_prepare_output_buffer:    test01.js:162
<glcolorbalance0> passthrough: reusing input buffer
▶0:00:05.492985000    42   0x1a67d0 DEBUG          basetransform gstbasetransform.c:2181:default_generate_output:          test01.js:162
<glcolorbalance0> using allocated buffer in 0x1ba6d0, out 0x1ba6d0
▶0:00:05.493235000    42   0x1a67d0 DEBUG          basetransform gstbasetransform.c:2192:default_generate_output:          test01.js:162
<glcolorbalance0> element is in passthrough
▶0:00:05.493479000    42   0x1a67d0 DEBUG          basetransform gstbasetransform.c:2383:gst_base_transform_chain:         test01.js:162
<glcolorbalance0> we have a pending DISCONT
▶0:00:05.493719000    42   0x1a67d0 TRACE      GST_REFCOUNTING gstobject.c:238:gst_object_ref:<sink:sink> 0x16bef8 ref   test01.js:162
1->2
▶0:00:05.493979000    42   0x1a67d0 TRACE      GST_REFCOUNTING gstobject.c:238:gst_object_ref:<sink> 0x16ba98 ref 6->7   test01.js:162
```

First results: in node.js

```
jl@carbonX1:~/w/github/fluendo-wasm/test$ node --experimental-wasm-threads --experimental-wasm-simd test01.js
0:00:00.003384315     42   0x103fb8 DEBUG            GST_MEMORY gstallocator.c:613:_priv_gst_allocator_initialize: memory alignment: 7
0:00:00.004818418     42   0x103fb8 DEBUG            GST_MEMORY gstallocator.c:589:gst_allocator_sysmem_init: init allocator 0x119ff0
0:00:00.005207646     42   0x103fb8 DEBUG            GST_MEMORY gstallocator.c:231:gst_allocator_register: registering allocator 0x119ff0 with name "SystemMemory"
0:00:00.005654194     42   0x103fb8 INFO               GST_INIT gstmessage.c:129:_priv_gst_message_initialize: init messages
0:00:00.006022190     42   0x103fb8 DEBUG      GST_ELEMENT_PADS gstelement.c:316:gst_element_base_class_init: type GstElement : factory 0
0:00:00.006450542     42   0x103fb8 DEBUG      GST_ELEMENT_PADS gstelement.c:316:gst_element_base_class_init: type GstBin : factory 0
0:00:00.007172155     42   0x103fb8 INFO               GST_INIT gstcontext.c:86:_priv_gst_context_initialize: init contexts
0:00:00.007575816     42   0x103fb8 INFO      GST_PLUGIN_LOADING gstplugin.c:324:_priv_gst_plugin_initialize: registering 0 static plugins
0:00:00.009468948     42   0x103fb8 DEBUG            GST_REGISTRY gstregistry.c:594:gst_registry_add_feature:<registry0> adding feature 0x125fe0 (bin)
0:00:00.009799991     42   0x103fb8 DEBUG       GST_REFCOUNTING gstobject.c:709:gst_object_set_parent:<bin> set parent (ref and sink)
0:00:00.010058352     42   0x103fb8 DEBUG      GST_ELEMENT_PADS gstelement.c:316:gst_element_base_class_init: type GstPipeline : factory 0x1263e8
0:00:00.010314381     42   0x103fb8 DEBUG            GST_REGISTRY gstregistry.c:594:gst_registry_add_feature:<registry0> adding feature 0x1263e8 (pipeline)
0:00:00.010536494     42   0x103fb8 DEBUG       GST_REFCOUNTING gstobject.c:709:gst_object_set_parent:<pipeline0> set parent (ref and sink)
0:00:00.010741755     42   0x103fb8 INFO      GST_PLUGIN_LOADING gstplugin.c:232:gst_plugin_register_static: registered static plugin "staticelements"
0:00:00.010957432     42   0x103fb8 DEBUG            GST_REGISTRY gstregistry.c:478:gst_registry_add_plugin:<registry0> adding plugin 0x125b30 for filename "(NULL)"
0:00:00.011208207     42   0x103fb8 INFO      GST_PLUGIN_LOADING gstplugin.c:234:gst_plugin_register_static: added static plugin "staticelements", result: 1
0:00:00.011543969     42   0x103fb8 INFO             GST_REGISTRY gstregistry.c:1837:ensure_current_registry: reading registry cache: /home/web_user/.cache/gstreamer-1.0/registry.wasm32.bin
0:00:00.012110564     42   0x103fb8 INFO             GST_REGISTRY gstregistrybinary.c:601:priv_gst_registry_binary_read_cache: Unable to mmap file /home/web_user/.cache/gstreamer-1.0/registry.wasm32.bin : Failed t
o open file "/home/web_user/.cache/gstreamer-1.0/registry.wasm32.bin": open() failed: No such file or directory
0:00:00.019389001     42   0x103fb8 INFO             GST_REGISTRY gstregistrybinary.c:611:priv_gst_registry_binary_read_cache: Unable to read file /home/web_user/.cache/gstreamer-1.0/registry.wasm32.bin : Failed t
o open file "/home/web_user/.cache/gstreamer-1.0/registry.wasm32.bin": No such file or directory
0:00:00.019714761     42   0x103fb8 DEBUG            GST_REGISTRY gstregistry.c:1865:ensure_current_registry: Updating registry cache
0:00:00.019890337     42   0x103fb8 INFO             GST_REGISTRY gstregistry.c:1704:scan_and_update_registry: Validating plugins from registry cache: /home/web_user/.cache/gstreamer-1.0/registry.wasm32.bin
0:00:00.020131055     42   0x103fb8 DEBUG            GST_REGISTRY gstregistry.c:1714:scan_and_update_registry: scanning paths added via --gst-plugin-path
0:00:00.020335431     42   0x103fb8 DEBUG            GST_REGISTRY gstregistry.c:1737:scan_and_update_registry: GST_PLUGIN_PATH not set
0:00:00.024373489     42   0x103fb8 DEBUG            GST_REGISTRY gstregistry.c:1749:scan_and_update_registry: GST_PLUGIN_SYSTEM_PATH not set
0:00:00.024724784     42   0x103fb8 DEBUG            GST_REGISTRY gstregistry.c:1756:scan_and_update_registry: scanning home plugins /home/web_user/.local/share/gstreamer-1.0/plugins
0:00:00.024953012     42   0x103fb8 DEBUG            GST_REGISTRY gstregistry.c:1417:gst_registry_scan_path_internal:<registry0> scanning path /home/web_user/.local/share/gstreamer-1.0/plugins
0:00:00.025468760     42   0x103fb8 DEBUG            GST_REGISTRY gstregistry.c:1421:gst_registry_scan_path_internal:<registry0> registry changed in path /home/web_user/.local/share/gstreamer-1.0/plugins: 0
0:00:00.025698966     42   0x103fb8 WARN             GST_REGISTRY gstregistry.c:1630:priv_gst_get_relocated_libgstreamer: Don't know how to retrieve the location of the shared library libgstreamer-1.0
0:00:00.025907472     42   0x103fb8 DEBUG            GST_REGISTRY gstregistry.c:1772:scan_and_update_registry: using plugin dir /home/jl/w/github/fluendo-wasm/build/target/lib/gstreamer-1.0
0:00:00.026139616     42   0x103fb8 DEBUG            GST_REGISTRY gstregistry.c:1417:gst_registry_scan_path_internal:<registry0> scanning path /home/jl/w/github/fluendo-wasm/build/target/lib/gstreamer-1.0
0:00:00.026609454     42   0x103fb8 DEBUG            GST_REGISTRY gstregistry.c:1421:gst_registry_scan_path_internal:<registry0> registry changed in path /home/jl/w/github/fluendo-wasm/build/target/lib/gstreamer-1
.0: 0
0:00:00.026846868     42   0x103fb8 DEBUG            GST_REGISTRY gstregistry.c:1651:gst_registry_remove_cache_plugins:<registry0> removing cached plugins
0:00:00.027060145     42   0x103fb8 INFO             GST_REGISTRY gstregistry.c:1796:scan_and_update_registry: Registry cache has not changed
0:00:00.027250260     42   0x103fb8 INFO             GST_REGISTRY gstregistry.c:1872:ensure_current_registry: registry reading and updating done
0:00:00.027445317     42   0x103fb8 INFO                GST_INIT gst.c:808:init_post: GLib runtime version: 2.76.0
0:00:00.028261220     42   0x103fb8 INFO                GST_INIT gst.c:810:init_post: GLib headers version: 2.76.0
0:00:00.028450266     42   0x103fb8 INFO                GST_INIT gst.c:811:init_post: initialized GStreamer successfully
```

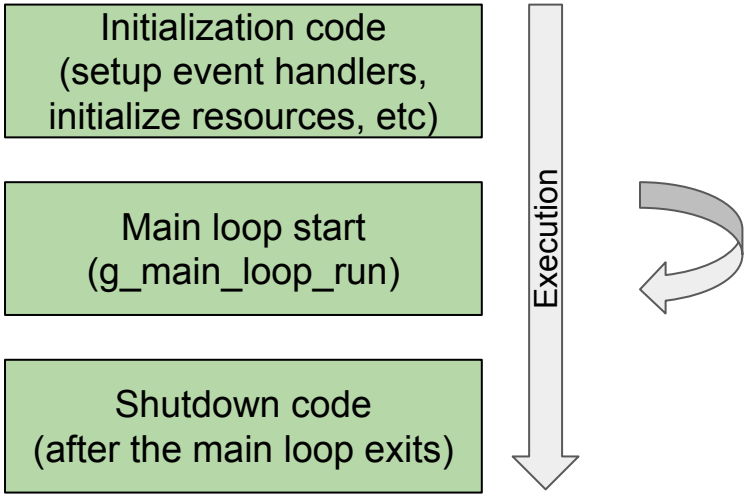
Missing video sink

- We wanted to show something
- Great support of SDL in Emscripten
- **No SDL sink in GStreamer**
- OpenGL/GLES/EGL maybe?

OpenGL

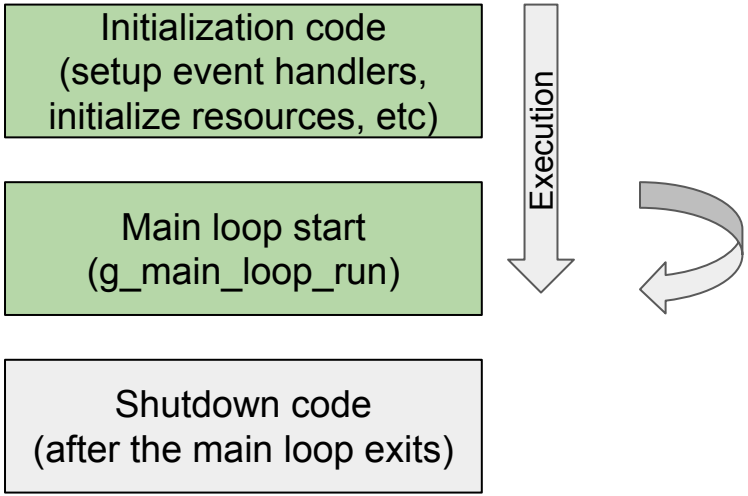
- Emscripten supports EGL and OpenGL (WebGL and GLES 1.0, 2.0)
- Compilation succeeded, execution failed
- EGL uses WebGL context behind the scenes and hardcodes:
  - Single static window -> No GstVideoOverlay
  - Overridable through JS initialization code
  - Redrawing is done automatically because *explicitSwapControl* is always set to false -> At requestAnimationFrame() freq.
- The actual threading model is, by default, EMSCRIPTEN_WEBGL_CONTEXT_PROXY_DISALLOW, meaning that all calls to OpenGL will happen on the calling thread and won't be proxied.
- **Threads blocked and no rendering at all**

Glib Main loop



- Kernel idles the process
- Wakes up on any fd event (select(), poll(), etc)
- GLib iterates (g_main_loop_iterate) and process the events that happened on the fds

Glib Main loop



- No select() or poll() in Emscripten
- Cooperative multi-tasking model
- Each page has a running slot
- A window can not hold the execution (sleep)
- **Lack of GWakeup**

- The execution is **unwound**
- Faked infinite loop
- requestAnimationFrame() -> g_main_context_iterate() (60 times per second)

OpenGL second try: new backend

- GStreamer's OpenGL-related elements calls happen on the same thread
- Emscripten supports different compilation and run-time features for the threading model
- New OpenGL "backend": WebGL
  - explicitSwapControl
  - renderViaOffscreenBackBuffer
  - proxyContextToMainThread
- **Only the main thread can access DOM elements or events**
- HTML5 events that can emulate a classic windowing system
- Web worker <-> Page communication is done through message passing, no shared memory.
- Transfer a canvas to a web worker (offscreen canvas) to draw with OpenGL

OpenGL second try: transfer canvas

- Emscripten owns the Web worker thread message handling: No message handling after the thread is created
- Emscripten has a new p_thread_attr to set the canvas to use.
- GLib owns the thread creation
- **New Glib API GThread * g_thread_emscripten_new (name, canvas, func, data)**
- **Still nothing to show in the screen**

Testing

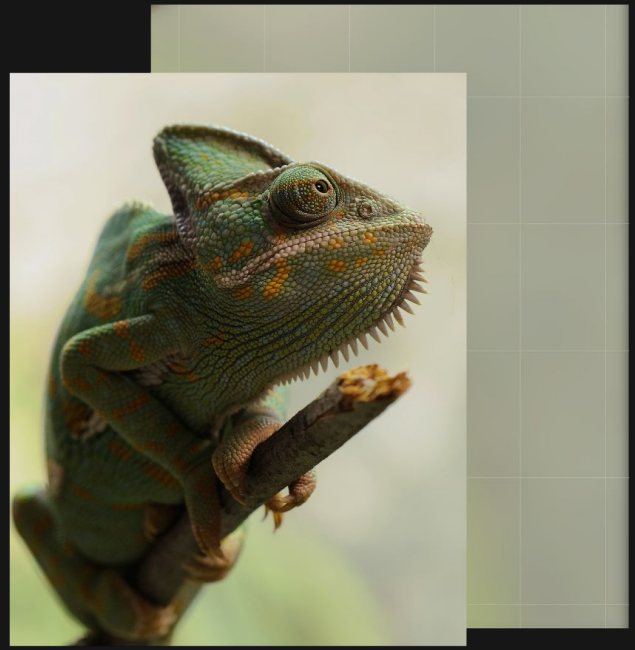
GstCheck

- It is mandatory to test
- Emscripten does not support fork()
- **Libcheck does not work without fork() support**
- **Tests that depend on dynamic loading of plugins, despite the configuration flags**

Test execution

- Test results:
  - **Ok: 44**
  - Expected Fail: 0
  - **Fail: 150**
  - Unexpected Pass: 0
  - Skipped: 0
  - Timeout: 16
- Good enough for us, the infrastructure was there.
- **How to handle testing on a non-dynamic or full-static compilation of GStreamer?**

WebCodecs replacement



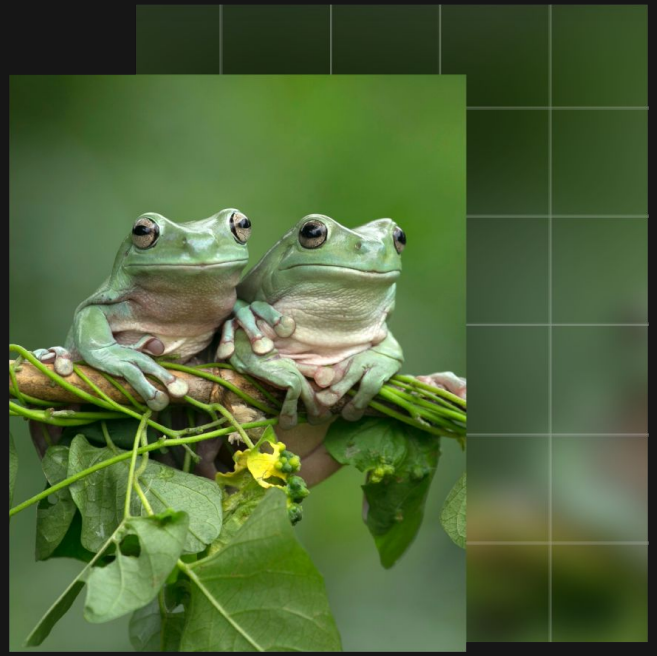
GStreamerWebCodecs.js

- Embind as an interface to call C++ code from Js
- Embind can expose classes
- Possibility to call Js code from C++
- Cannot call Javascript function from C++ from a secondary thread
- Javascript ArrayBuffer/UInt8ArrayBuffer/... matched as std::string
- Cannot extend an Embind class
- **We managed to override native Js classes with our own and call our C++ classes**

```
};

EMSCRIPTEN_BINDINGS(my_class_example)   {
  class_<AudioDecoder>("AudioDecoder")
    .constructor<val>()
    .property("state", &AudioDecoder::getState)
    .property("decodeQueueSize", &AudioDecoder::getDecodeQueueSize)
    .class_function("isConfigSupported", &AudioDecoder::IsConfigSupported)
    .function("close", &AudioDecoder::Close)
    .function("configure", &AudioDecoder::Configure)
    .function("decode", &AudioDecoder::Decode)
    .function("reset", &AudioDecoder::Reset)
    .function("flush", &AudioDecoder::Flush)
```

Contributing and future



All open source

- Repositories
  - GLib: https://github.com/fluendo/glib/tree/wasm-vips-2.76.0
  - GStreamer: https://github.com/fluendo/gstreamer/tree/wasm-1.22

Future

- Opportunity to bring **GStreamer to other domains**, and browser-based multimedia processing is a niche worth exploring
- This is just the tip of the iceberg
- We can not foresee the future usage of this:
  - Plug&Play Browser's interfaces?
  - New encoders for the browser
  - New decryptors by replacing EME as a GStreamer backend (Adding MSE and EME to GStreamer presentation)
  - Missing protocols in the browser?
  - GStreamer.js?
  - What else?
- We need more feedback from the different actors to see what this can become

Thank you!

Questions?